

Chapitre 2

Éléments de base du langage Java

Règles de base

- ❑ Java est sensible à la casse
- ❑ Les blocs de code sont encadrés par des accolades **{ }**
- ❑ Chaque instruction se termine par un point virgule **;**
- ❑ Une instruction peut tenir sur plusieurs lignes:

- Exemple :

- ❑ l'instruction « `int x=3;` » peut être écrite :

```
int
x
=
3;
```

Identificateurs

- ❑ Chaque objet, classe, programme ou variable est associé à un nom : **un identificateur**.
- ❑ Un identificateur peut se composer de tous les **caractères alphanumériques** et des caractères **_** et **\$**.
- ❑ Le premier caractère doit être une lettre, **_** ou **\$**.
- ❑ Un identificateur ne peut pas être un mot réservé du langage Java (ex: **abstract**, **new**, **return**, **try**, ...)

Commentaires

- ❑ Ils sont sautés par le compilateur (pas de ';' à la fin)
- ❑ Il existe trois types de commentaires en Java :
 - 1. Monoligne.** Exemple : `int N=1; // déclaration du compteur`
 - 2. Multiligne.** Exemple : `/* commentaires ligne 1
commentaires ligne 2 */`
 - 3. De documentation automatique.**

Exemple :

```
/**  
 * commentaire de la méthode  
 * @param val la valeur à traiter  
 * @return Rien  
 * @deprecated Utiliser la nouvelle méthode XXX  
 */
```

Variables

- ❑ Une variable possède un **nom**, un **type** et une **valeur**.
- ❑ Pour utiliser une variable il faut la déclarer pour lui réserver de la mémoire avant de lui affecter une valeur.
- ❑ Exemples :
 - `int x, y, somme=0;`
 - `String nom;`
 - `LocalDate dateNaissance;`
- ❑ Remarque :
 - Une variable n'est visible que dans le bloc où elle est déclarée.
 - Elle peut être simple (`int`, `long`,...) ou objet (`String`, `LocalDate`,...)

Types primitifs

- ❑ Ont une taille identique quelque soit la plate-forme d'exécution (d'où l'indépendance aux plateformes d'exécution)

Entiers

Type	Désignation	Longueur	Valeurs
byte	octet signé	8 bits	-128 à 127
short	entier court signé	16 bits	-32768 à 32767
int	entier signé	32 bits	-2147483648 à 2147483647
long	entier long	64 bits	-9223372036854775808 à 9223372036854775807

Types primitifs

Réels

Type	Désignation	Longueur	Valeurs
float	virgule flottante simple précision (IEEE754)	32 bits	1.401e-045 à 3.40282e+038
double	virgule flottante double précision (IEEE754)	64 bits	2.22507e-308 à 1.79769e+308

Autre

Type	Désignation	Longueur	Valeurs	Commentaires
boolean	valeur logique	1 bit	true ou false	pas de conversion possible vers un autre type
char	caractère Unicode	16 bits	\u0000 à \uFFFF	entouré de cotes simples dans du code Java

Types primitifs et Wrappers

- ❑ Pour chaque type primitif, dispose d'une classe équivalente appelée *wrapper* ou *Enveloppe* pour le traiter.
- ❑ Ces classes sont :
 - **Character** : pour le type **char**
 - **Boolean** : pour le type **boolean**
 - **Byte** : pour le type **byte**
 - **Short** : pour le type **short**
 - **Integer** : pour les valeurs entières (**int**)
 - **Long** : pour les entiers long signés (**long**)
 - **Float** : pour les nombres à virgules flottante (**float**)
 - **Double** : pour les nombres à virgule flottante en double précision (**double**)

Types primitifs et Wrappers

□ Exemples :

■ Classe Character :

- **charValue()** : retourne le caractère primitif.
- **static boolean isDigit(char ch)** : vérifie si le caractère est un chiffre.
- **static boolean isLetter(char ch)** : vérifie si c'est une lettre.
- **static boolean isLetterOrDigit(char ch)** : lettre ou chiffre.
- **static char toUpperCase(char ch)** : majuscule.
- **static char toLowerCase(char ch)** : minuscule.
- **static boolean isWhitespace(char ch)** : vérifie si espace blanc, tabulation, saut de ligne,....

Types primitifs et Wrappers

■ Classe Integer :

- **intValue()** : retourne la valeur primitive.
- **static Integer valueOf(int i/String s)** : retourne un objet Integer correspondant à l'int ou la chaîne
- **static int parseInt(String s)** : convertit une chaîne en int.
- **String toString()** : retourne la valeur sous forme de chaîne.
- **static String toBinaryString(int i)** : retourne la représentation binaire.
- **static int compare(int x, int y)** : compare deux entiers et retourne -1, 0 ou 1.
- **static Integer max(int a, int b)** : retourne le plus grand des deux.
- **static Integer min(int a, int b)** : retourne le plus petit des deux.
- **static int sum(int a, int b)** : retourne la somme de deux entiers.

Types primitifs et Wrappers

- ❑ Le passage du type primitif au **Wrapper** est appelé **Boxing** alors que le contraire est appelé **UnBoxing**
- ❑ Ces deux opérations se font automatiquement.
- ❑ Exemple :

```
// Déclaration d'un int et conversion en Integer (Autoboxing)
int nombre = 42;
Integer wrappedInt = nombre; // Autoboxing (int → Integer)

// Conversion d'un Integer en int (Unboxing)
int unboxedInt = wrappedInt; // Unboxing (Integer → int)
```

Format des types primitifs

- ❑ **Le format des nombres entiers :**
 - Un nombre de type **byte**, **short**, **int** et **long** peuvent être codés en **décimal**, **hexadécimal** (commence par **0x**) ou **octal** (commencer par **0**).
 - Le suffixe **l** ou **L** permet de spécifier que c'est un entier **long**.
 - Exemple: `long x=584756L`
- ❑ **Le format des nombres décimaux :**
 - Les types **float** et **double** stockent des nombres flottants.
 - Ils doivent posséder soit un point, un exposant ou l'un des suffixes **f**, **F**, **d**, **D**.
 - Exemples : **float** `pi = 3.14f`; **double** `v = 3d` ; **float** `f = +.1f` , `d = 5e3f`; `//5 x 103`

Format des types primitifs

❑ Remarque :

- Il est possible de préciser des nombres qui n'ont pas le partie entière ou décimale.
- Par défaut un littéral est de type `double` : pour définir un `float` il faut le suffixer par la lettre `f` ou `F`.
- Exemple : `double w = 1.1;`
- **Attention !** `float pi = 3.14;` // provoque une **erreur** à la compilation
- Solution : `float pi = 3.14f;`

Format des types primitifs

❑ Le format des caractères :

- Un caractère Unicode est codé sur **16 bits**
- Il doit être **entouré** par des **apostrophes**. Exemple : `char rep = 'A';`
- Une valeur de type `char` peut être considérée comme un **entier** non négatif de **0** à **65535**.
- Les caractères 0 à 255 correspondent à ASCII
- La conversion implicite par affectation n'est pas possible.

❑ Liste des caractères Unicode:

```
for(int i=0;i<Character.MAX_VALUE;i++)  
    System.out.println(i+" : "+(char)i);
```

Format des types primitifs

❑ Extrait de l'exécution :

- ...
- ا : 1575
- ب : 1576
- ...
- ج : 1580
- ح : 1581
- ...

❑ Exemple : la lettre arabe JIM (ج)

- `System.out.println((char)1580); //décimal`
- `System.out.println("\u062C"); // hexadécimal`

Chaînes de caractères

- ❑ Une chaîne de caractères stocke une séquence de caractères alphanumériques.
- ❑ Définition d'une chaîne : `String texte = "bonjour";`
- ❑ Le compilateur Java remplace les constantes chaînes par des objets de type `String`.
- ❑ Exemple :
 - `String texte2 = texte.substring(0,3); // renvoie la chaine bon`
 - `String res2 = "tata".replace('a','o'); // renvoie la chaine toto`

Chaînes de caractères

❑ Quelques méthodes de la classe String:

- `int length()` : Retourne la longueur de la chaîne.
- `String subString(int debut, int fin)` : découpe et renvoie la chaîne de l'index début à fin
- `String toUpperCase()` : transforme notre chaîne en majuscule.
- `String toLowerCase()` : transforme notre chaîne en minuscule.
- `int compareTo(String chaîne2)` : Compare deux chaînes alphabétiquement.
 - ❑ Si le résultat `< 0` : notre chaîne précède chaîne2
 - ❑ Si le résultat `> 0` : notre chaîne2 précède notre chaîne
 - ❑ Si le résultat `= 0` : les deux chaînes sont égales

❑ Exemple : `String ch1="Ali", ch2="Bachir"`

- `ch1.compareTo(ch2)` retourne un entier **négatif** car « Ali » précède « Bachir » en ordre alphabétique.

Chaînes de caractères

❑ Remarque :

- La classe String est immuable : toute modification passe par la création d'une nouvelle chaîne.
- Pour modifier le contenu d'une chaîne de caractère sans créer une nouvelle, utilisez la classe `StringBuilder` (ou `StringBuffer` dans le cas de threads)
- ❑ Exemple :

```
StringBuilder sb = new StringBuilder("Bonjour ");
sb.append("monde"); // Bonjour monde
sb.insert(7, " le"); // Bonjour le monde
sb.delete(8, 16); // Bonjour
sb.reverse(); //ruojnob
System.out.println(sb.toString());
```

Chaînes de caractères

□ Les caractères spéciaux dans les chaînes:

- Apostrophe : \'
- Guillemet : \"
- Anti-slash : \\
- Tabulation : \t
- retour arrière (backspace) : \b
- retour chariot : \r
- saut de page (form feed) : \f
- saut de ligne (newline) : \n
- caractère ASCII ddd (octal) : \Oddd
- caractère ASCII dd (hexadécimal) : \xdd
- caractère Unicode dddd (hexadécimal) : \udddd

Chaînes de caractères

□ Exemples:

- Pour afficher **Bonjour "Ali"**
→ `System.out.println("Bonjour \"Ali\" ");`
- Pour afficher **Bonjour**
Ali
→ `System.out.println("Bonjour \n Ali ");`
- Pour afficher **c:\programmes\java**
→ `System.out.println("c:\\programmes\\java");`

Opérateurs

- Les opérateurs arithmétiques se notent :
 - + (addition)
 - - (soustraction)
 - * (multiplication)
 - / (division)
 - % (reste de la division)
- Ils peuvent se combiner à l'opérateur d'affectation.

Opérateurs

- **Remarque :**
 - L'opérateur + permet de concaténer une chaîne de caractères avec d'autres chaînes de caractères ou types élémentaires.
 - Exemple :
 - 1+2 donne 3
 - "1" + "2" donne "12"
 - "1" + 2 donne "12"

Affectation

- Syntaxe : **variable = expression.**
- Association de droite à gauche :
 - Exemple : **x = y = z = 0;**
- Liste d'opérateurs abrégée d'affectation :

Opérateur	Exemple	Signification
=	a=10	
+=	a+=10	a = a + 10
-=	a-=10	a = a - 10
=	a=10	a = a * 10
/=	a/=10	a = a / 10
%=	a%=10	Reste de la division
^=	a^=10	a = a ^ 10

Java - Dr A. Belangour

25

Comparaisons

- Java propose des opérateurs pour toutes les comparaisons :
 - **>** (strictement supérieur)
 - **<** (strictement inférieur)
 - **>=** (supérieur ou égal)
 - **<=** (inférieur ou égal)
 - **==** (égalité)
 - **!=** (différent de)
 - **&&** (ET logique)
 - **||** (OU logique)
 - **!** (Négation)
 - **&** (ET binaire)
 - **^** (OU exclusif binaire)
 - **|** (OU binaire)
 - **?:** (opérateur conditionnel)
 - **Exemple : a ? b : c**
équivalent à : si a alors b
sinon c
 - **Remarque :** b et c doivent
retourner le même type

Java - Dr A. Belangour

26

Comparaisons

□ Priorités des opérateurs du plus au moins prioritaire :

- 1) les parenthèses : **()**
- 2) les opérateurs d'incrémentation : **++ , --**
- 3) les opérateurs de multiplication, division, et modulo : ***, / , %**
- 4) les opérateurs d'addition et soustraction : **+, -**
- 5) les opérateurs de décalage : **<< et >>**
- 6) les opérateurs de comparaison : **< , > , <= , >=**
- 7) les opérateurs d'égalité : **== , !=**
- 8) l'opérateur OU exclusif : **^**
- 9) l'opérateur ET : **&**
- 10) l'opérateur OU : **|**

Comparaisons

- 11) l'opérateur ET logique : **&&**
- 12) l'opérateur OU logique : **||**
- 13) les opérateurs d'assignement : **= , += , -=**

□ Remarque 1:

- Les parenthèses ayant une forte priorité, l'ordre d'interprétation des opérateurs peut être modifié par des parenthèses.

Comparaisons

❑ Remarque 2 :

- L'opérateur **==** teste l'égalité entre les types primitifs comme les entiers, réels,...

- ❑ Exemple : `int x,y;`

```
....  
if (x==y) {...}
```

- Pour les classes la méthode `equals()` est à utiliser

- ❑ Exemple : `String ch1,ch2;`

```
....  
if (ch1.equals(ch2)){....}
```

Lecture & Ecriture des variables

❑ Ecriture : `System.out.println(expression)`

- Exemple : `System.out.println(" le résultat est " + R)`

❑ Lecture : Se fait grâce à la classe **Scanner**

- `next()` : permet de lire un mot de type **String**
- `nextLine()` : permet de lire une chaîne de caractère
- `nextInt()` : permet de lire un **int**
- `nextLong()` : permet de lire un **long**
- `nextDouble()` : permet de lire un **double**
- `nextFloat()` : permet de lire un **float**

Lecture & Ecriture des variables

□ Utilisation :

- 1) La classe doit être importée du package java.util
- 2) Elle doit être ensuite instanciée (création et initialisation de la variable)
- 3) Finalement ses méthodes de lecture peuvent être appelées

Lecture & Ecriture des variables

□ Exemple:

```
import java.util.Scanner;

public class SaisieClavier{

    public static void main(String [] args){
        Scanner s = new Scanner(System.in);
        System.out.println("Entrez un entier:");
        int n = s.nextInt();
        System.out.println(" Le carré de :"+ n +" est :"+ n*n);
        s.close();
    }
}
```

Opérations arithmétiques

□ On distingue entre deux arithmétiques :

- L'arithmétique entière
- L'arithmétique en virgule flottante

Opérations arithmétiques

□ **Arithmétique entière**

- Conversion implicite vers le type **int** des types numériques (promotion entière) pour renforcer la sécurité du code

■ **Exemple :**

```
short z , x = 5 , y = 15;  
z = x + y ; //provoque une erreur à la compilation  
Incompatible type for =. Explicit cast needed to  
convert int to short.  
z = x + y ; //erreur à la compilation ^ 1 error
```

■ **Explication :**

- Le résultat est promu en type int (32 bits) est affecté dans un type short (16 bits): **incompatibilité !!**

■ **Solution :** **cast** explicite : `z = (short) ( x + y );`

Opérations arithmétiques

- La division par zéro pour les types entiers lève l'exception `ArithmeticException`
- **Exemple**

```
/* test sur la division par zero de nombres entiers */  
class test3 {  
    public static void main (String args[]) {  
        int valeur=10;  
        double resultat = valeur / 0; // provoque exeption  
        System.out.println("Resultat = " + resultat);  
    }  
}
```

Opérations arithmétiques

- **L'arithmétique en virgule flottante**
 - la **division par zéro** des `float` ou `double`, ne produit pas d'exception mais une des 3 valeurs spéciales:
 1. **NaN (not a number** : nombre non défini):
 - `Float.NaN` ou `Double.NaN`
 2. **Infinity (+∞** : valeur positive supérieure au plafond du type):
 - `Float.POSITIVE_INFINITY` ou `Double.POSITIVE_INFINITY`
 3. **- Infinity (-∞** : valeur négative supérieure au plafond du type):
 - `Float.NEGATIVE_INFINITY` ou `Double.NEGATIVE_INFINITY`

Opérations arithmétiques

Tableau récapitulatif

X	Y	X/Y	X%Y
valeur finie	0	$\pm\infty$	NaN
valeur finie	$\pm\infty$	0	x
0	0	NaN	NaN
$\pm\infty$	valeur finie	$\pm\infty$	NaN
$\pm\infty$	$\pm\infty$	NaN	NaN

Exemple:

- Dans ce programme nous divisons délibérément un float par zéro. Remarquez le résultat !

Solution : `/* test sur la division par zéro de nombres flottants */`

```
class test2 {  
    public static void main (String args[]) {  
        float valeur=10f;  
        double résultat = valeur / 0;  
        System.out.println(" Resultat = " + résultat);  
    }  
}
```

Java - Dr A. Belangour

37

Opérations arithmétiques

L'incrément et la décrémentation

- Les opérateurs d'incrément et de décrémentation sont : `++n`, `n++`, `--n`, `n--`
- L'opérateur `++` (resp. `--`) renvoie la valeur avant incrément et après décrémentation s'il est postfixé et après incrément et après décrémentation s'il est préfixé.
- Exemple :

`y=x++` ⇔ `y=x; x = x + 1;`

`y=++x` ⇔ `x = x + 1; y=x;`

Java - Dr A. Belangour

38

Opérations arithmétiques

❑ **Exercice** : Quel sera le résultat de l'exécution du programme :

```
public class test4 {  
    public static void main (String args[]) {  
        int a=0,b=0;  
        System.out.println("a = " + a + " b = " + b);  
        a=b++;  
        System.out.println("a = " + a + " b = " + b);  
        a=++b;  
        System.out.println("a = " + a + " b = " + b);  
        a=a++;  
        System.out.println("a = " + a + " b = " + b);  
    }  
}
```

Opérations arithmétiques

❑ **Résultat:**

■ int a=0; int b=0; ➡ a=0 b=0

■ a=b++; ➡ a=0 b=1

■ a=++b; ➡ a=2 b=2

■ a=a++; ➡ a=2 b=2



➡ **attention** : a ne change pas de valeur

■ Par contre l'instruction "a=++a" incrémente la valeur de a

Structures de contrôles

- ❑ Sont similaires aux autres langages de programmation.
- ❑ Sont composés de :
 1. **boucles** : permettent d'itérer sur un certain nombre d'instructions (while, do..while, for, for pour collections)
 2. **branchements conditionnels** : Permettent d'effectuer des tests (if, opérateur ternaire, switch)

Structures de contrôles

- ❑ Boucle **WHILE**:

```
while ( condition ) {  
    ...  
}
```

- ❑ Boucle **DO..WHILE**:

```
do {  
    ...  
} while ( condition )
```

- **Similitudes** : Dans les deux boucles le code est exécuté tant que la condition est vraie. Lorsque la condition passe à faux le bloc du **While** est sauté.
- **Différence** : Dans **do...While**, la boucle est au moins exécutée une fois quelque soit la valeur de la condition;

Structures de contrôles

□ Boucle FOR normale :

■ Syntaxe : `for (initialisation; condition; modification) {...}`

■ Exemple :

```
for (int i =0; i<10; i++) {  
    System.out.println(" le carré de"+i+ "est :"+i*i);  
}
```

□ Boucle for des collections :

■ Syntaxe : `for (type variable : collection)`

□ Exemple :

```
int[] t= {23,1,12,33,41,57,16,27,81,19};  
for (int x : t) {  
    System.out.println(x);  
}
```

Structures de contrôles

□ Les branchements conditionnels Se composent :

- Du mot clé **if**
- De l'opérateur ternaire
- Du mot clé **switch**

Structures de contrôles

□ Tests avec IF :

- Forme :

```
if (condition) { ... }  
else if (condition) { ... }  
else { ... }
```

- Exemple :

```
if (moy > 10)  
    resultat = "V";  
else  
    resultat = "RAT";
```

□ L'opérateur ternaire:

- Forme :

`condition ? valeur-vrai : valeur-faux`

- Facilité d'écriture pour un if avec un else.

- Exemple :

```
resultat = moy >= 10 ? "V" : "RAT";
```

```
System.out.println(moy >= 10 ? "V" : "RAT");
```

Structures de contrôles

□ Remarque :

- Il ne faut jamais déclarer une variable à l'intérieur d'une boucle ou d'une condition
!!

Structures de contrôles

□ Tests avec Switch :

- Switch permet d'effectuer des tests pour des valeurs constantes de type (`byte`, `short`, `int`, `char`, `String`).
- Forme :
`switch` (expression)
{ `case` constante1 : instr11; instr12; `break`;
 `case` constante2 : ...
 `default` : ...
}
- En l'absence du `break`, l'exécution passe au case suivant.
- Il est possible d'imbriquer des `switch`

Java - Dr A. Belangour

47

Structures de contrôles

□ Exemple :

```
import java.util.Scanner;
public class SwitchDemo1 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Fournir un nombre de 1 à 12 !");
        int n = s.nextInt();
        String mois;
        switch (n) {
            case 1: mois="Janvier"; break;
            case 2: mois="Février"; break;
            ...
            case 12: mois="Décembre"; break;
            default: mois="Mois Invalide";break;
        }
        System.out.println(mois);
    }
}
```

Java - Dr A. Belangour

48

Structures de contrôles

- ❑ Remarque : il est possible de regrouper des valeurs qui ont les mêmes traitement.
- ❑ Exemple :
 - Transformer l'exemple précédent pour indiquer la saison du mois entré.

Structures de contrôles

```
import java.util.Scanner;
public class SwitchDemo2 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Fournir un nombre de 1 à 12 !");
        int n = s.nextInt();
        String saison;
        switch (n) {
            case 12 :
            case 1 :
            case 2 : saison = "Hiver"; break;
            case 3 :
            case 4 :
            case 5 : saison = "Printemps"; break;
            case 6 :
            case 7 :
            case 8 : saison = "Eté"; break;
            case 9 :
            case 10 :
            case 11 : saison = "Automne"; break;
            default : saison = "Mois Invalide";
        }
        System.out.println(saison);
    }
}
```


Structures de contrôles

□ Nouvelle syntaxe de switch

- Utilisation d'une flèche « -> » et suppression du « **break** »
- Utilisation de la virgule « , » en cas de plusieurs valeurs
- Permet d'assigner directement le résultat de chaque cas à une variable.
- Renvoi de valeur automatique dans le cas d'une seule instruction
- Renvoi avec le mot clé « **yield** » dans le cas de plusieurs instructions

Structures de contrôles

□ Exemple 1 :

```
import java.util.Scanner;
public class SwitchDemo1 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Fournir un nombre de 1 à 12 !");
        int n = s.nextInt();
        String mois = switch (n) {
            case 1 -> "Janvier"; //retourne janvier
            case 2 -> "Février";
            ...
            case 12 -> "Décembre";
            default -> "Mois Invalide";
        }
        System.out.println(mois);
    }
}
```

Structures de contrôles

❑ Exemple 1 v2:

```
import java.util.Scanner;
public class SwitchDemo1 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Fournir un nombre de 1 à 12 !");
        int n = s.nextInt();
        System.out.println(switch (n) {
            case 1 -> "Janvier";
            case 2 -> "Février";
            ...
            case 12 -> "Décembre";
            default -> "Mois Invalide";
        });
    }
}
```

Structures de contrôles

❑ Exemple 2 : regroupement des valeurs qui ont les mêmes traitement.

```
import java.util.Scanner;
public class SwitchDemo2 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Fournir un nombre de 1 à 12 !");
        int n = s.nextInt();
        System.out.println(switch (n) {
            case 12, 1, 2 -> "Hiver"; //renvoi Hiver
            case 3, 4, 5 -> "Printemps";
            case 6, 7, 8 -> "Eté";
            case 9, 10, 11 -> "Automne";
            default -> "Mois Invalide";
        });
    }
}
```

Structures de contrôles

❑ Remarque :

- Dans le cas de plusieurs instructions après la flèche :
 - ❑ Mettre des accolades
 - ❑ Utiliser le mot clé « yield » pour renvoyer le résultat
- Syntaxe : `case valeur -> {instructions;...; yield resultat;}`
- Exemple : `case 3 -> {int x=3, y=5; yield x+y;}`

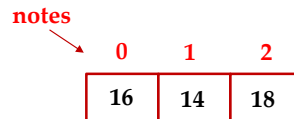
Tableaux

- ❑ Les tableaux sont des structures de données regroupant plusieurs valeurs de même type.
- ❑ On peut accéder à n'importe quelle valeur à partir de son rang ou indice.
- ❑ Exemples de déclarations:
 - `float[] notes = new float[3] // tableau de 3 réels`
 - `int[] t=new int[10] // tableau de 10 entiers`
 - `String[] noms= new String[20] // tableau de 20 strings`
- ❑ Remarque :
 - Une fois fixée, la taille d'un tableau ne peut changer.

Tableaux

❑ Exemple d'affectation :

- `notes[0]=16`
- `notes[1]=14`
- `notes[2]=18`



❑ Taille du tableau : propriété **length**

- Exemple : **notes.length** renvoie **3**

❑ Parcours :

- Requière une boucle avec un seul indice de 0 jusqu'à `length-1`

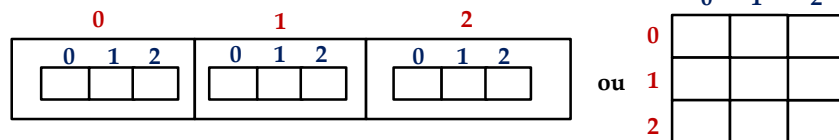
Tableaux : Exemple

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Combien d'éléments voulez vous ?");
        int n = s.nextInt(); //lecture de la taille souhaitée
        int[] t = new int[n]; // déclaration et réservation
        for (int i = 0; i < t.length; i++) {
            System.out.println("Entrer Element " + i + " : ");
            t[i] = s.nextInt();
        }
        System.out.println("Affichage du Contenu du tableau :");
        for (int i = 0; i < t.length; i++) {
            System.out.println("t[" + i + "] = " + t[i]);
        }
    }
}
```

Tableaux

- ❑ Les tableaux à deux dimensions sont déclarés comme étant un tableau de tableaux.

- Exemple : `float[][] t = new float[3][3];`



- ❑ Taille du tableau : **length**

- **t.length** renvoie la taille de la 1ère dimension
- **t[0].length, t[1].length,...** renvoient la taille de la 2ème dimension

Tableaux

- ❑ Parcours :

- Une première boucle pour parcourir les cases de la première dimension (indice de 0 jusqu'à length-1)
- Une deuxième boucle pour parcourir les contenus des cases qui sont des tableaux

- Forme :

```
for (int i = 0; i < t.length; i++) {  
    for (int j = 0; j < t[i].length; j++) {  
        // traitement sur t[i][j]  
    }  
}
```

```
t[0][0]; t[0][1]; t[0][2];  
t[1][0]; t[1][1]; t[1][2];  
t[2][0]; t[2][1]; t[2][2];
```

Tableaux

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println(" Taille de la première dimension?");
        int n = s.nextInt(); //lecture de la taille de la 1ere dim
        System.out.println(" Taille de la deuxième dimension ?");
        int m = s.nextInt(); //lecture de la taille de la 2eme dim
        int[][] t = new int[n][m]; // déclaration et réservation
        // parcours et remplissage du tableau
        for (int i = 0; i < t.length; i++) {
            for (int j = 0; j < t[i].length; j++) {
                System.out.println("Entrer Element " + i + " , " + j + " : ");
                t[i][j] = s.nextInt();
            }
        }
    }
}
```

Java - Dr A. Belangour

61

Tableaux

```
// parcours et affichage du contenu du tableau
System.out.println(" Affichage du contenu du tableau");
for (int i = 0; i < t.length; i++) {
    for (int j = 0; j < t[i].length; j++) {
        System.out.println("t["+i+"]["+j+"] = "+ t[i][j]);
    }
}
} // fin main
} // fin classe
```

Java - Dr A. Belangour

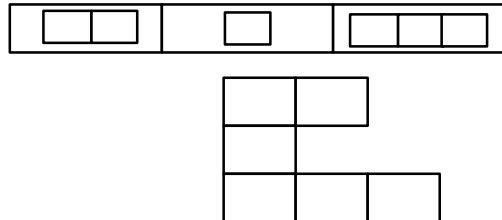
62

Tableaux

- ❑ Remarque : La taille des tableaux de la seconde dimension peut ne pas être identique pour chaque occurrence.

- ❑ Exemple :

- `int[][] t= new int[3][];`
- `t[0] = new int[2];`
- `t[1] = new int[1];`
- `t[2] = new int[3];`



- ❑ Parcours :

```
for (int i = 0; i < t.length; i++) {
    for (int j = 0; j < t[i].length; j++) {
        // traitement sur t[i][j]
    }
}
```

Tableaux

- ❑ Chaque élément du tableau est initialisé selon son type par l'instruction `new` :

- ❑ `0` pour les numériques,
- ❑ `'\0'` pour les caractères,
- ❑ `false` pour les booléens et
- ❑ `null` pour les chaînes de caractères et les autres objets.

- ❑ L'initialisation explicite d'un tableau. Exemple :

- ❑ `int[] tableau = {10,20,30,40,50};`
- ❑ `int[][] tableau = {{5,1},{6,2},{7,3}};`
- ❑ `int[][] tableau = {{4,2},{9},{6,3,4}};`

- ❑ La taille du tableau est déduite automatiquement

Tableaux

- ❑ La boucle for des collections : permet d'itérer sur un tableau ou une collection par leurs contenus

- ❑ Exemple :

```
public class TestForArray {  
    public static void main(String[] args) {  
        int[] t1= {23,1,12,33,41,57,16,27,81,19};  
        for (int x: t1) { System.out.println(x); }  
        int[][] t2={{8,3,1},{4,5,7,12},{7,6},{9,15,87,19}};  
        for (int[] tab : t2) {  
            for (int x : tab) {  
                System.out.print(x+" ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Java - Dr A. Belangour

65

Tableaux

- ❑ Exemple de déclaration de tableaux dans une méthode :

- En paramètre : public void afficher(int[] t){ ... }
- En retour : public int[] calculer(){ ... }

- ❑ Exemples d'appel :

- int[] t = {5,2,9};
- e.afficher(t);
- ou
- e.afficher(new int[] {5,2,9});

- ❑ Remarque :

- Les tableaux sont toujours transmis par référence puisque ce sont des objets.

Java - Dr A. Belangour

66

Conversions de types

- ❑ On peut avoir besoin de convertir une variable d'un type vers un autre type.
- ❑ Deux cas de figure existent :
 - Conversion vers un type de taille plus grande : se font sans perte d'information, et sans besoin d'être spécifié.
 - Conversion vers un type de taille plus petite: peut entraîner une perte d'information, et doit être spécifiées par un opérateur de *cast*.
 - ❑ Exemple:
 - `int` entier = 5;
 - `float` flottant = (`float`) entier;

Conversions de types

- ❑ Conversion d'une chaîne de caractère en :
 - `int` : méthode `Integer.parseInt(chaine-à-converter)`
 - `long` : méthode `Long.parseLong(chaine-à-converter)`
 - `float` : méthode `Float.parseFloat(chaine-à-converter)`
 - `double` : méthode `Double.parseDouble(chaine-à-converter)`
 - ...
- ❑ Exemple :
 - ❑ `String ch="10"; int x=Integer.parseInt(ch)`
- ❑ Remarque : L'instruction de conversion doit être obligatoirement être mise dans un bloc try/catch

Conversions de types

❑ Exemple :

```
try {  
    x=Integer.parseInt(ch);  
}  
catch (Exception e) {  
    System.out.println("erreur : " + e.getMessage());  
}
```

Conversions de types

❑ Conversion d'un **int/long/float/double** en chaîne de caractère String:

- Première méthode : le concaténer avec une chaîne vide

❑ Exemple :

- **int** x=10;
- **String** ch="" + x;

- Deuxième méthode : utiliser la méthode `valueOf` de `String`

❑ Exemple :

- **int** y = 10;
- **String** montexte = **new** `String()`;
- montexte =montexte.**valueOf**(y);

Méthodes

- ❑ Une méthode est un sous-programme qui permet d'effectuer une tâche.
- ❑ Elle est composée d'une signature et d'un corps.
 - La signature est constituée de:
 - ❑ Une visibilité
 - ❑ Un type de retour
 - ❑ Un nom
 - ❑ En ensemble de paramètres
 - Le corps est un bloc de code délimité par des accolades

Méthodes

- ❑ Une méthode retourne le résultat grâce à **return**
- ❑ Les instructions après return sont ignorées.
- ❑ Exemple 1:

```
public class Main {  
    // déclaration  
    public static int carré(int x) { return x*x; }  
  
    public static void main(String[] args) {  
        int nombre=3;  
        //appel de la méthode  
        int résultat = carré(nombre);  
        System.out.println("le carré de "+nombre+" est :"+ résultat);  
    }  
}
```

Méthodes

- ❑ Si la méthode ne retourne rien, alors on utilise **void**.

- ❑ Exemple 2:

```
public class Main {  
    // déclaration  
    public static void carré(int x) {  
        int résultat = x*x;  
        System.out.println("le carré de "+x+" est :"+ résultat);  
    }  
  
    public static void main(String[] args) {  
        int nombre=3;  
        //appel de la méthode  
        carré(nombre);  
    }  
}
```

- ❑ **Remarque :**

- l'instruction *return* dans une méthode de type *void* permet de quitter la méthode.

Méthode main

- ❑ Constitue le point d'entrée pour l'exécution
- ❑ **Signature :** `public static void main (String args[]) {..}`
- ❑ Cette signature doit être respectée sinon la machine virtuelle ne la reconnaitra pas.
- ❑ Le tableau « args » permet de recueillir des arguments en ligne de commande.

Méthode main

- ❑ Faire un programme qui calcule la moyenne de notes fournies en ligne de commande.
- ❑ Remarques:
 - Pour exécuter cet exemple , il faut se mettre en mode commande (ms dos)
 - Et taper par exemple :
 - ❑ `java Moyenne 10 12 14 14 10`
 - Le résultat sera affiché sous dos aussi comme suit :
 - ❑ `la moyenne est : 12.0`

Méthode main

```
public class Moyenne{
    public static void main(String[] args) {
        double somme=0,moyenne,note;
        try{
            for (String arg : args){
                note=Double.parseDouble(arg);
                somme=somme+note;
            }
            moyenne=somme/args.length;
            System.out.println("la moyenne est : "+moyenne);
        }
        catch (FormatException e){
            System.out.println("Erreur : "+e.getMessage());
        }
    }
}
```

Enumérations

- ❑ Lorsqu'une variable varie dans un ensemble de valeurs prédéfinis, il faut créer un type énumération.
- ❑ Une énumération est un ensemble fini de constantes numériques (correspondant à 0, 1, 2, ...)
- ❑ Exemple :
 - `public enum Jours { LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE }`
 - `Jours jr=Jours.VENDREDI; // exemple de variable`
- ❑ Le nom de l'énumération précède toujours ses constantes (sauf dans Switch)

Java - Dr A. Belangour

77

Enumérations : Exemple

```
import java.util.Scanner;
public class Main {
    public enum Saisons { automne, hiver, printemps, été}
    public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        System.out.println("saisir une saison :");
        String rep=s.next().toLowerCase();
        try {
            Saisons sz = Saisons.valueOf(rep);
            System.out.println(switch (sz) {
                case automne -> "Les feuilles tombent";
                case hiver -> " Il fait Froid";
                case printemps -> "il fait beau";
                case été -> "Il fait chaud"; });
        } catch (IllegalArgumentException e) {System.out.println("Saison invalide "); }
    }
}
```

Java - Dr A. Belangour

78

Dates & Temps

- ❑ Les classes gérant la date et le temps font partie du package **java.time**
- ❑ Principales Classes :
 - **LocalDate** : Représente une date sans heure.
 - **LocalTime** : Représente une heure sans date.
 - **LocalDateTime** : Combine date et heure.
 - **ZonedDateTime** : Intègre un fuseau horaire aux dates et temps.
 - **DateTimeFormatter** permet de formater la date et le temps
 - **Period** : permet de représenter une période en années, mois et jours.
 - **Duration** : permet de représenter une durée en heures, minutes, secondes et nanosecondes.

Dates & Temps : classe LocalDate

- ❑ Méthodes de la classe LocalDate :
 - **now()** : Renvoie l'instant actuel sous forme de LocalDate.
 - ❑ **Exemple** : `LocalDate date = LocalDate.now();`
 - **of(int year, int month, int dayOfMonth)** : Crée une instance de LocalDate avec l'année, le mois et le jour spécifiés.
 - ❑ Exemple : `LocalDate date = LocalDate.of(2024, 2, 18);`
 - **plusDays(long daysToAdd) / minusDays(long daysToSubtract)** : Ajoute/soustrait un nombre spécifié de jours à/from une date.
 - ❑ **Exemple** : `LocalDate futureDate = date.plusDays(7);`
 - ❑ `LocalDate pastDate = date.minusDays(7);`

Dates & Temps : classe LocalDate

- `isBefore(LocalDate other)` / `isAfter(LocalDate other)` : Compare deux dates pour déterminer si l'une est antérieure ou postérieure à l'autre.
 - Exemple : `boolean isBefore = date.isBefore(futureDate);`
 - Exemple : `boolean isAfter = date.isAfter(pastDate);`

Dates & Temps : classe LocalTime

- Méthodes de la classe `java.time.LocalTime` :
 - `now()` : Renvoie l'instant actuel sous forme de `LocalTime`.
 - **Exemple** : `LocalTime time = LocalTime.now();`
 - `of(int hour, int minute)` : Crée une instance de `LocalTime` avec l'heure et la minute spécifiées.
 - **Exemple** : `LocalTime time = LocalTime.of(12, 30);`
 - `plusHours(long hoursToAdd)` / `minusHours(long hoursToSubtract)` : Ajoute/soustrait un nombre spécifié d'heures à/from une heure.
 - **Exemple** : `LocalTime futureTime = time.plusHours(3);`
 - **Exemple** : `LocalTime pastTime = time.minusHours(2);`

Dates & Temps : classe LocalDateTime

❑ Méthodes de la classe java.time.LocalDateTime :

- `now()` : Renvoie l'instant actuel sous forme de LocalDateTime.
 - ❑ Exemple : `LocalDateTime dateTime = LocalDateTime.now();`
- `of(int year, int month, int dayOfMonth, int hour, int minute)` : Crée une instance de LocalDateTime avec l'année, le mois, le jour, l'heure et la minute spécifiés.
 - ❑ Exemple : `LocalDateTime dateTime = LocalDateTime.of(2022, 1, 18, 12, 30);`
- `plusDays(long daysToAdd) / minusDays(long daysToSubtract)` : Ajoute/soustrait un nombre spécifié de jours à une date et heure.
 - ❑ **Exemple** : `LocalDateTime futureDateTime = dateTime.plusDays(7);`
 - ❑ **Exemple** : `LocalDateTime pastDateTime = dateTime.minusDays(7);`

Dates & Temps : classe ZonedDateTime

❑ Méthodes de la classe java.time.ZonedDateTime

- `now()` : Renvoie l'instant actuel sous forme de ZonedDateTime en tenant compte du fuseau horaire système.
 - ❑ Exemple : `ZonedDateTime zonedDateTime = ZonedDateTime.now();`
- `of(int year, int month, int dayOfMonth, int hour, int minute, int second, int nanoOfSecond, ZoneId zone)` : Crée une instance de ZonedDateTime avec l'année, le mois, le jour, l'heure, la minute, la seconde, le nanoseconde et le fuseau horaire spécifiés.

Dates & Temps : classe ZonedDateTime

- Exemple : `ZonedDateTime zonedDateTime = ZonedDateTime.of(2022, 1, 18, 12, 30, 0, 0, ZoneId.of("Africa/Casablanca"));`
- `withZoneSameInstant(ZoneId zone)` / `withZoneSameLocal(ZoneId zone)` : Ajuste le fuseau horaire d'une `ZonedDateTime` tout en maintenant l'instant (`withZoneSameInstant`) ou en gardant la date et l'heure locales (`withZoneSameLocal`).
- Exemple : `ZonedDateTime newZoneInstant = zonedDateTime.withZoneSameInstant(ZoneId.of("America/New_York"));`
- Exemple : `ZonedDateTime newZoneLocal = zonedDateTime.withZoneSameLocal(ZoneId.of("Asia/Tokyo"));`

Dates & Temps : classe DateTimeFormatter

- Méthodes de la classe `DateTimeFormatter` :
 - `ofPattern(String pattern)` : Crée un formateur de date personnalisé en utilisant un motif spécifié.
 - Exemple : `DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");`
 - `format(TemporalAccessor temporal)` : Convertit un objet `LocalDate` (resp. `LocalDateTime`, `ZonedDateTime`) en une chaîne de caractères
 - Exemple : `LocalDateTime dateTime = LocalDateTime.now();`
`String formattedDateTime = formatter.format(dateTime);`

Dates & Temps : classe DateTimeFormatter

- `parse(CharSequence text)` : Convertit une chaîne de caractères en un objet TemporalAccessor en utilisant le format spécifié par le DateTimeFormatter.

□ Exemple :

```
String dateString = "18-01-2022 14:30:00";
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
LocalDateTime parsedDateTime = LocalDateTime.parse(dateString, formatter);
```

Dates & Temps : Exemple

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
public class GestionTempsExemple {
    public static void main(String[] args) {
        // Obtention de la date et de l'heure actuelles
        LocalDateTime maintenant = LocalDateTime.now();
        // Création d'un événement futur dans 7 jours
        LocalDateTime evenementFutur = maintenant.plusDays(7);
        // Formatage de la date et de l'heure actuelles
        DateTimeFormatter formateur = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
        String maintenantFormate = formateur.format(maintenant);
        System.out.println("Date et heure actuelles : " + maintenantFormate);
        // Formatage de l'événement futur
        String evenementFuturFormate = formateur.format(evenementFutur);
        System.out.println("Événement futur dans 7 jours : " + evenementFuturFormate);
        // Calcul de la différence entre la date actuelle et l'événement futur
        long joursJusquaEvenement = maintenant.until(evenementFutur).toDays();
        System.out.println("Jours jusqu'à l'événement : " + joursJusquaEvenement + " jours");
    }
}
```

Dates & Temps : classe Period

- ❑ Utilisée pour représenter une période en années, mois et jours.
- ❑ Méthodes principales :
 - `of(int years, int months, int days)`: Crée une période spécifique.
 - `between(LocalDate start, LocalDate end)`: Calcule la période entre deux dates.
 - `plusYears(long yearsToAdd), plusMonths(long monthsToAdd), plusDays(long daysToAdd)`: Ajoute des valeurs à la période.
 - `getYears(), getMonths(), getDays()`: Récupère les composantes de la période.
- ❑ Exemple :

```
LocalDate start = LocalDate.of(2020, 1, 1);  
LocalDate end = LocalDate.of(2025, 3, 10);  
Period period = Period.between(start, end);  
System.out.println(period);
```

Java - Dr A. Belangour

89

Dates & Temps : classe Duration

- ❑ Utilisée pour représenter une durée en heures, minutes, secondes et nanosecondes.
- ❑ Méthodes principales :
 - `ofHours(long hours), ofMinutes(long minutes), ofSeconds(long seconds)`: Crée une durée spécifique.
 - `between(Temporal startInclusive, Temporal endExclusive)`: Calcule la durée entre deux instants.
 - `plusHours(long hoursToAdd), plusMinutes(long minutesToAdd), plusSeconds(long secondsToAdd)`: Modifie la durée.
 - `toHours(), toMinutes(), toSeconds()`: Convertit en différentes unités.

Java - Dr A. Belangour

90

Dates & Temps : classe Duration

□ Exemple :

```
LocalTime start = LocalTime.of(10, 30);  
LocalTime end = LocalTime.of(12, 45);  
Duration duration = Duration.between(start, end);  
System.out.println(duration.toHours()); // 2  
System.out.println(duration.toMinutes()); // 135
```